POLITECNICO DI MILANO

Parallel Programming: Design Guidelines

058165 - PARALLEL COMPUTING

Fabrizio Ferrandi

Politecnico di Milano
Dipartimento di Elettronica, Informazione e Bioingegneria
fabrizio.ferrandi@polimi.it

Parallel Algorithms and Parallel Programs

- Designing parallel *algorithms* is not an easy task
- There is not an algorithm to design parallel algorithms
- These are some rules which can help in the design
- Design of parallel *programs* have more specific rules but they depend on the particular chosen language/architecture

Parallel Algorithms and Parallel Programs

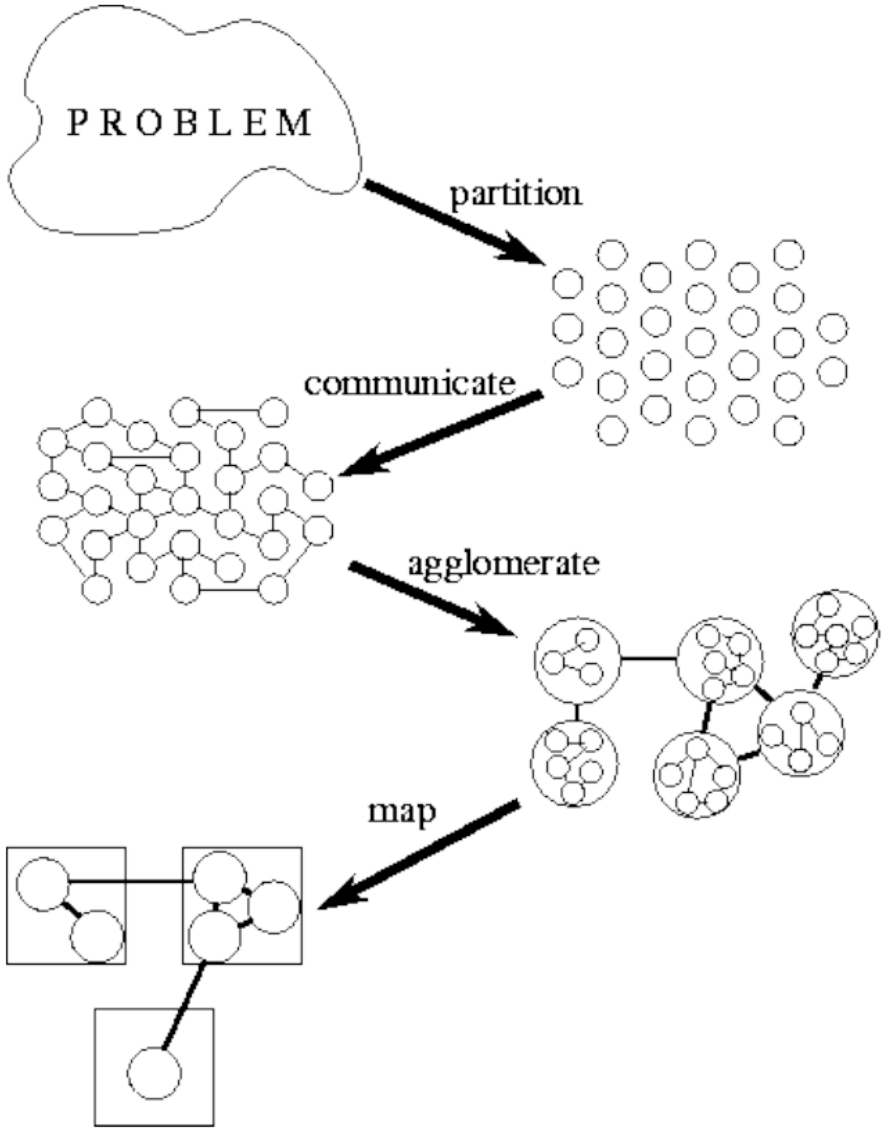
- Most problems have several possible parallel solutions
- A good approach is to start from machine-independent issues (concurrency) and delay target-specific aspects as much as possible

Parallel Algorithms and Parallel Programs

1. Design a parallel algorithm:
   - Understand the problem to be solved
   - Analyze data dependences
   - Partition the solution
2. Design a parallel program:
   - Analyze the target architecture(s)
   - Select best parallel programming language
   - Analyze the communications (cost, latency, bandwidth, visibility, synchronization, etc.)

- => PCAM methodology

PCAM Design Methodology



- Partitioning => task/data decomposition
- Communication => task execution coordination
- Agglomeration => evaluation of the structure
- Mapping => resource assignment

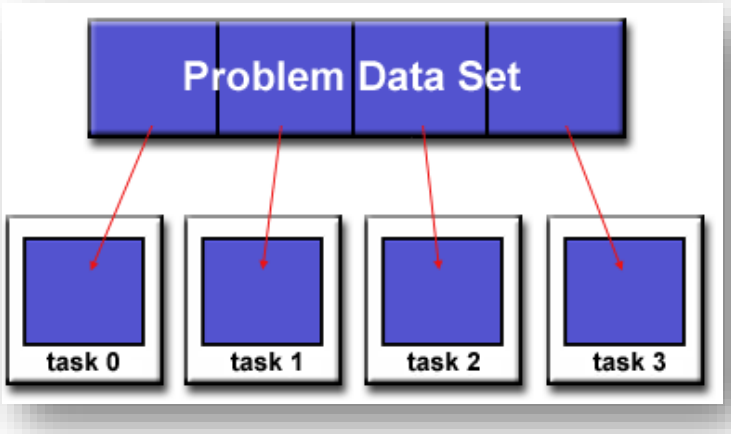
- In practice, these steps are often overlapping

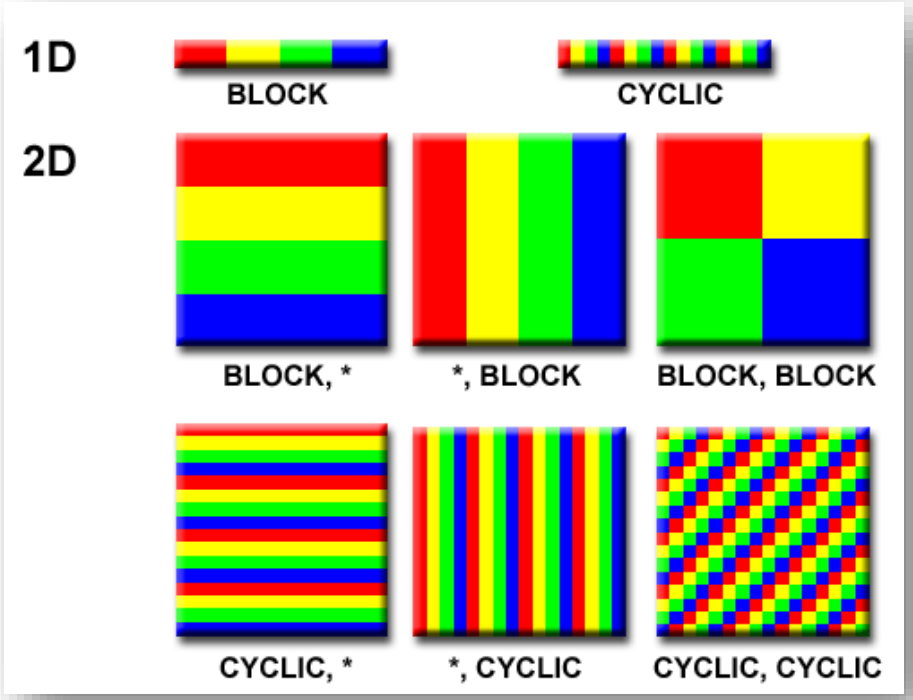
Partitioning

- Partitioning stage is intended to expose opportunities for parallel execution in the given problem
- Focus on defining *large number of small tasks* to yield a fine-grained decomposition of the problem
- A good partition divides into small pieces both the *computational tasks* associated with a problem and the *data on which the tasks operate*, without replications
- There are two approaches to partitioning:
  - Domain decomposition
  - Functional decomposition
- Mixing domain/functional decomposition is possible

Domain (data) Decomposition

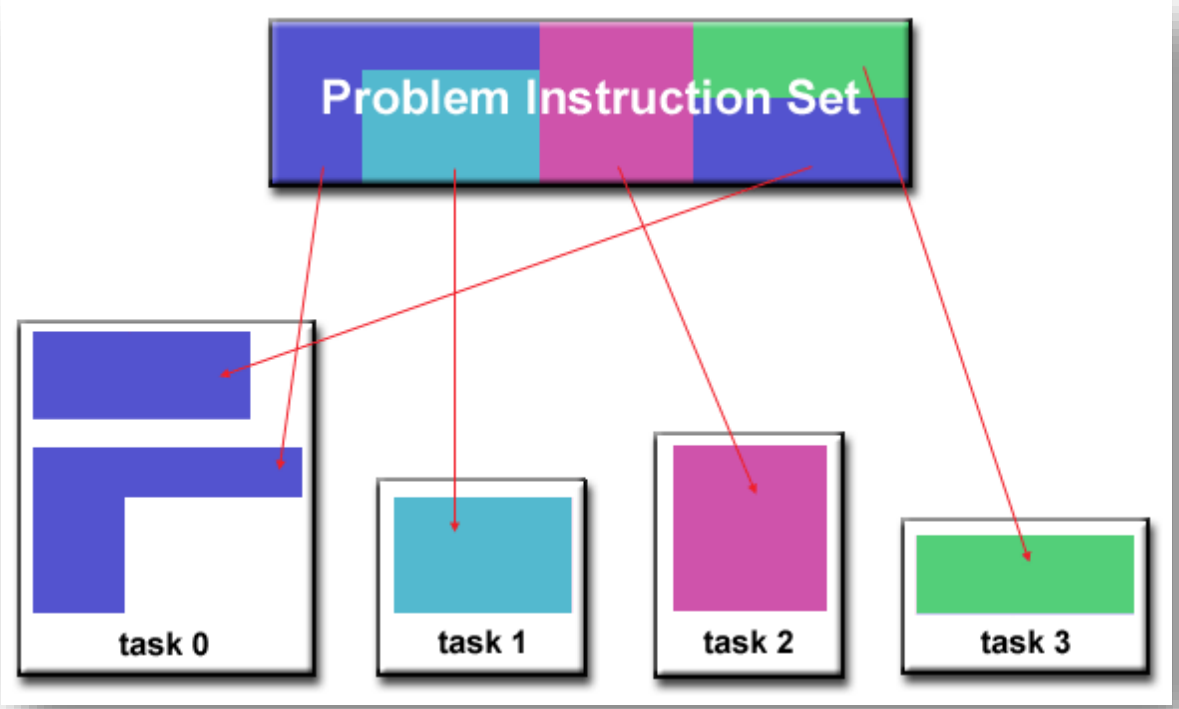
- The data associated with a problem is decomposed
- Each parallel task then works on a portion of the data
- When possible, portions should be small and of equal size





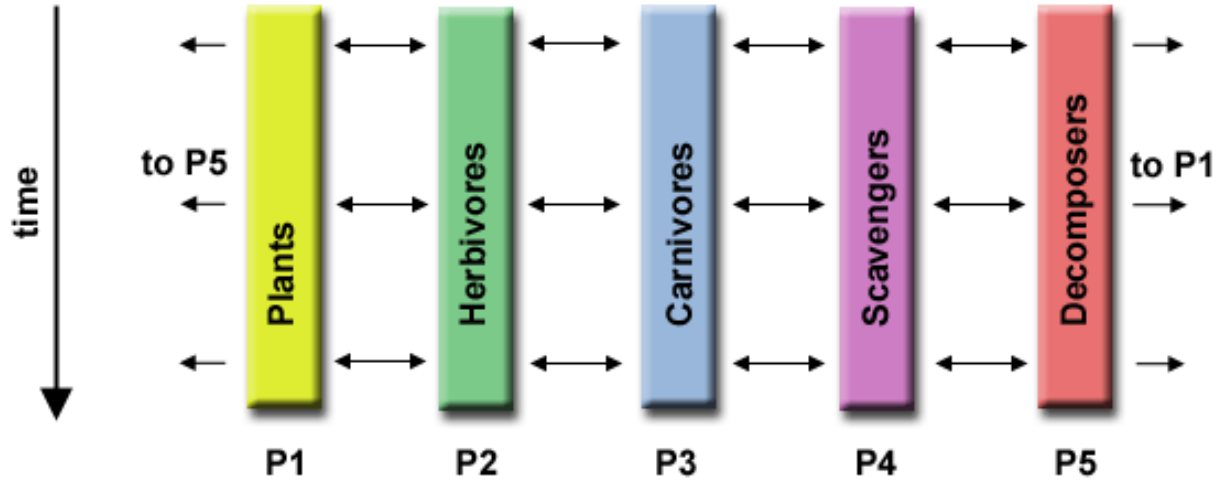
Functional Decomposition

- The focus is on the computation that is to be performed rather than on the data manipulated by the computation
- The problem is decomposed according to the work that must be done



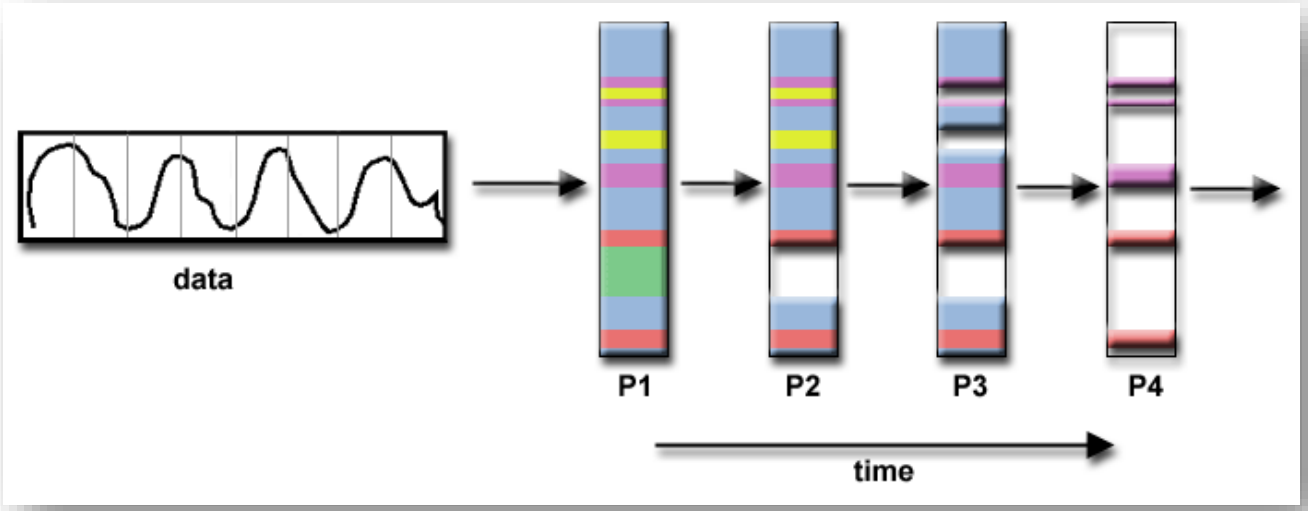
Functional Decomposition: Ecosystem Modeling

- Each task calculates the population of a group, whose growth depends on that of its neighbors
- As time progresses, each process calculates its current state, then exchanges information with the neighbor populations
- All tasks then progress to calculate the state at the next time step



Functional Decomposition: Signal Processing

- An audio signal data set is passed through four distinct computational filters
- The first segment of data must pass through the first filter before progressing to the second. When it does, the second segment of data passes through the first filter. By the time the fourth segment of data is in the first filter, all four tasks are busy



Partitioning

- Checklist:
  - Does your partition define at least an order of magnitude more tasks than there are processors in your target computer? If not, may lose design flexibility.
  - Does your partition avoid redundant computation and storage requirements? If not, may not be scalable.
  - Are tasks of comparable size? If not, it may be hard to allocate each processor equal amounts of work.
  - Does the number of tasks scale with problem size? If not, may not be able to solve larger problems with more processors
  - Have you identified several alternative partitions?

Communication

- Tasks generated by a partition **must** interact to allow the computation to proceed
  - Information flow: channel and message
- Types of communication
  - Local vs. Global: small set of neighbors or many tasks (possibly at the same time)
  - Structured vs. Unstructured: regular (tree, grid) or arbitrary communication patterns
  - Static vs. Dynamic: identity of communication partners can be determined by runtime conditions

Communication

They mainly depend on the characteristics of the target architecture

Visibility:

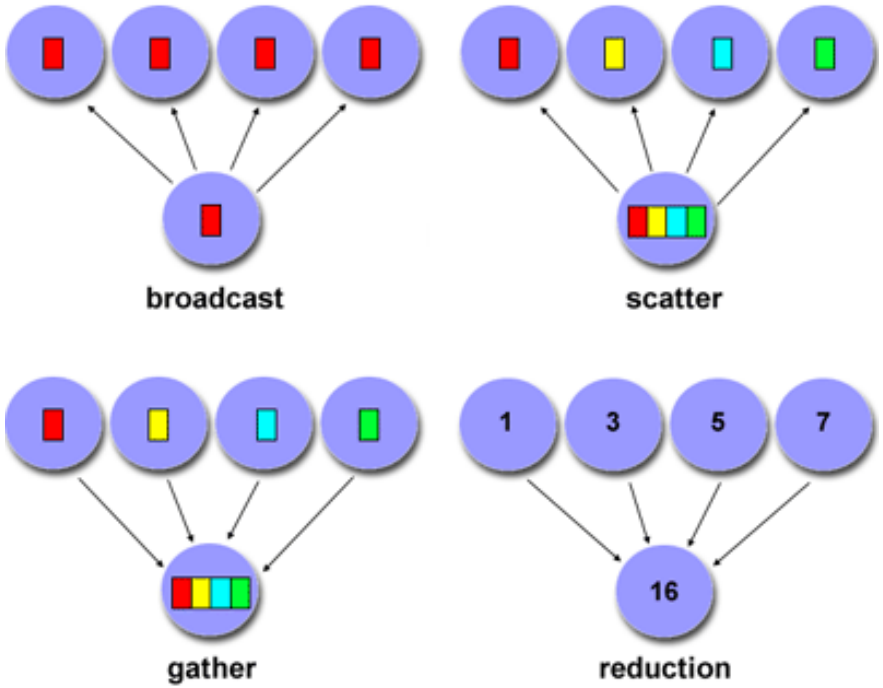
- Explicit communication (e.g., message passing)
- Implicit communication (e.g., shared memory)

Synchronization

- Synchronous (i.e., requiring handshaking and blocking)
- Asynchronous (tasks transfer data independently)

Communication

- Point-to-point - involves two tasks with one task acting as the sender/producer of data, and the other acting as the receiver/consumer. Unfeasible for global communications!
- Collective - involves data communication between more than two tasks, which are often specified as being member of a common group, or collective.



Communication

- When we move to the architecture-dependent implementation phases, communication is probably the most critical aspect
- Communication overhead cost can nullify the benefits of the parallelization itself
- Overhead is due to:
  - Transmission
  - Synchronization
- Try to minimize the number of channels and organize communication to allow concurrent execution
  - Overlap communication and computation

Communication

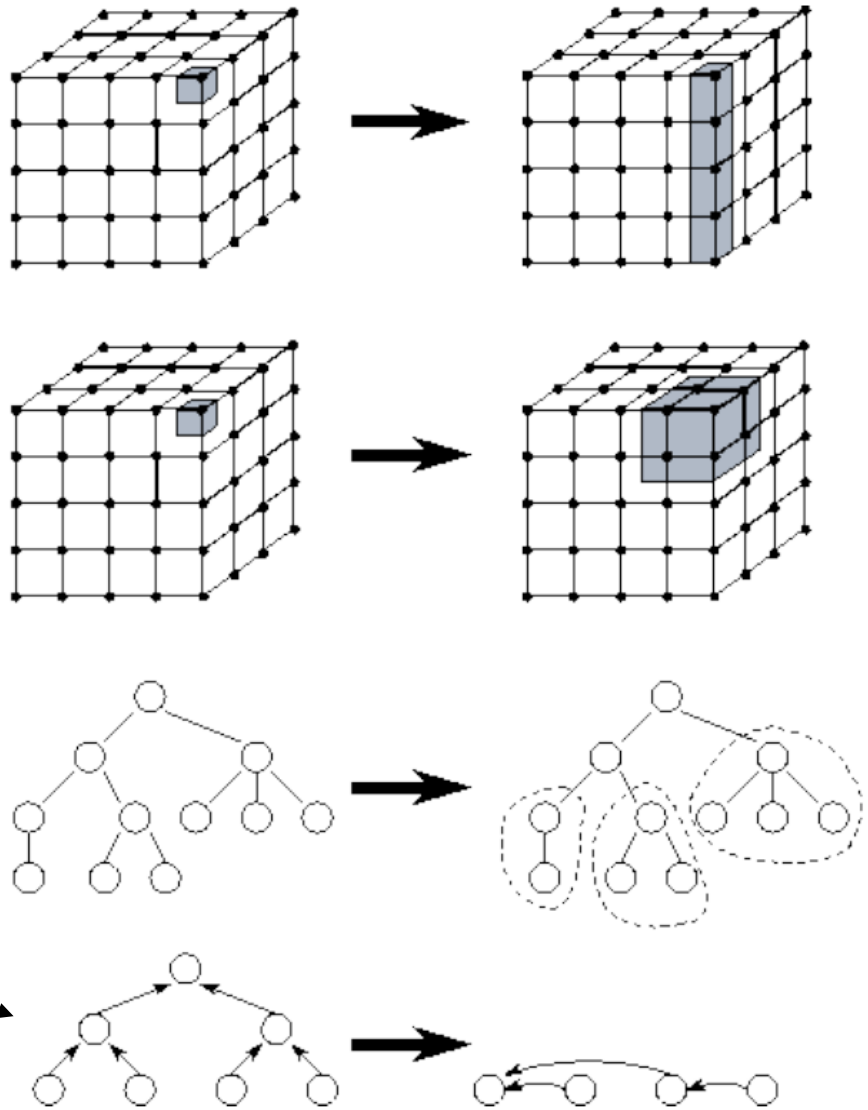
- Checklist:
  - Is the distribution of communications equal among tasks? Unbalanced communication may limit scalability
  - What is the communication locality? Many point-to-point communications are expensive
  - What is the degree of communication concurrency? Communication operations may be parallelized
  - Is computation associated with different tasks able to proceed concurrently? Can communication be overlapped with computation? Try to reorder computation and communication to expose opportunities for parallelism

Agglomeration

- Move from parallel abstractions to real implementation
- Revisit partitioning and communication to get efficiency *on a particular parallel architecture*
- Reduce number of tasks, combine them into larger ones (increased *granularity*, reduced *communication*)
- Replicate data and/or computation if it can be useful
- Maintain flexibility to allow scalability, portability and future mapping decisions
  - Load balancing
  - Computation/communication overlapping

Agglomeration

- Agglomeration examples:
  - reduce the dimension of the partitioning
  - combine adjacent tasks
  - coalesce sub-trees
  - combine nodes



Agglomeration

- Agglomeration changes important algorithm and performance ratios
  - Consider that also task creation has an overhead
  - Reduce the number of messages where the size of transferred data can't be reduced
- **Surface-to-volume**: increase in task size, reduction in communication
  - Communication/computation ratio decreases
  - Efficiency increases
  - Parallelism decreases
  - Agglomerate in *all dimensions*!
  - Always beneficial if there are tasks that cannot execute concurrently

Agglomeration

- Checklist:
  - Has increased locality reduced communication costs?
  - Is replicated computation worth it? Does data replication compromise scalability?
  - Are tasks still balanced in terms of computation and communication?
  - Does the number of tasks still scale with problem size?
  - Is there still sufficient concurrency?
  - Is there room for more agglomeration?

Mapping

- Specify *where* each task shall be executed
  - Less of a concern on shared-memory systems
- Attempt to minimize execution time
  - Place concurrent tasks on different processors to enhance physical concurrency (parallelism)
  - Place communicating tasks on same processor, or on processors close to each other, to increase locality
  - Strategies can conflict!
  - Possible resource limitations

Mapping

- Mapping problem is NP-complete, heuristics needed
- Load balancing (partitioning) algorithms
  - Static or dynamic
- Task-based (task scheduling) algorithms
  - Used when functional decomposition yields many tasks with weak locality requirements
  - Use task assignment to keep processors busy computing
  - Consider centralized and decentralized schemes

Mapping

- Checklist:
  - Is static mapping too restrictive and non-responsive?
  - Is dynamic mapping too costly in overhead?
  - Does centralized scheduling lead to bottlenecks?
  - Do dynamic load-balancing schemes require too much coordination to re-balance the load?
  - What is the tradeoff of dynamic scheduling complexity versus performance improvement?
  - Are there enough tasks to achieve high levels of concurrency? If not, processors may idle.

Sequential Version of Fibonacci

- Fibonacci sequence is defined as:

$$F(n) = \begin{cases} 0 & n = 0 \\ 1 & n = 1. \\ F(n-1) + F(n-2) & n > 1 \end{cases}$$

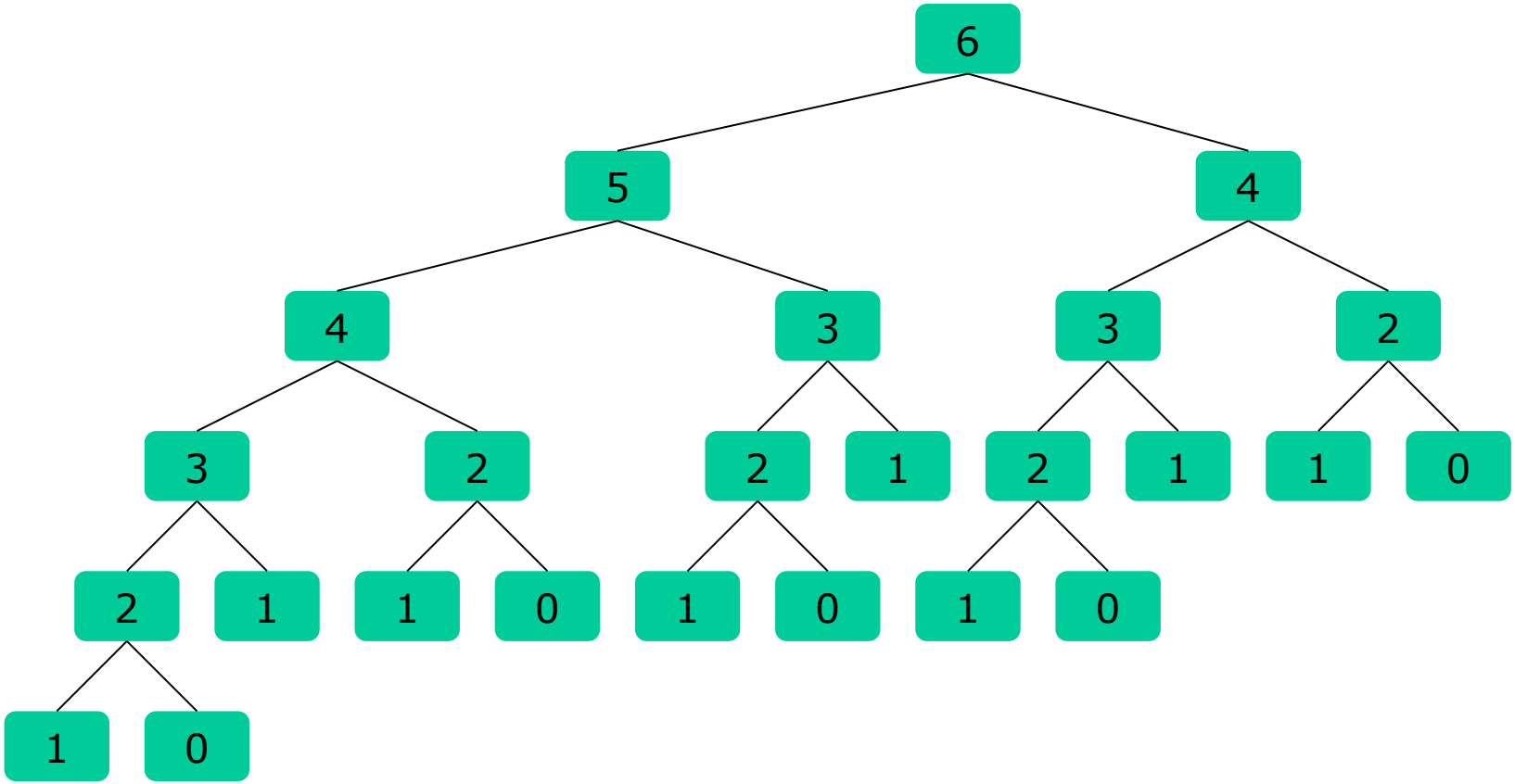
0,1,1,2,3,5,8,13,21,...

```
int fib (int n){
  if(n<2)
    return n;
  return fib(n-1)+fib(n-2);
}
```

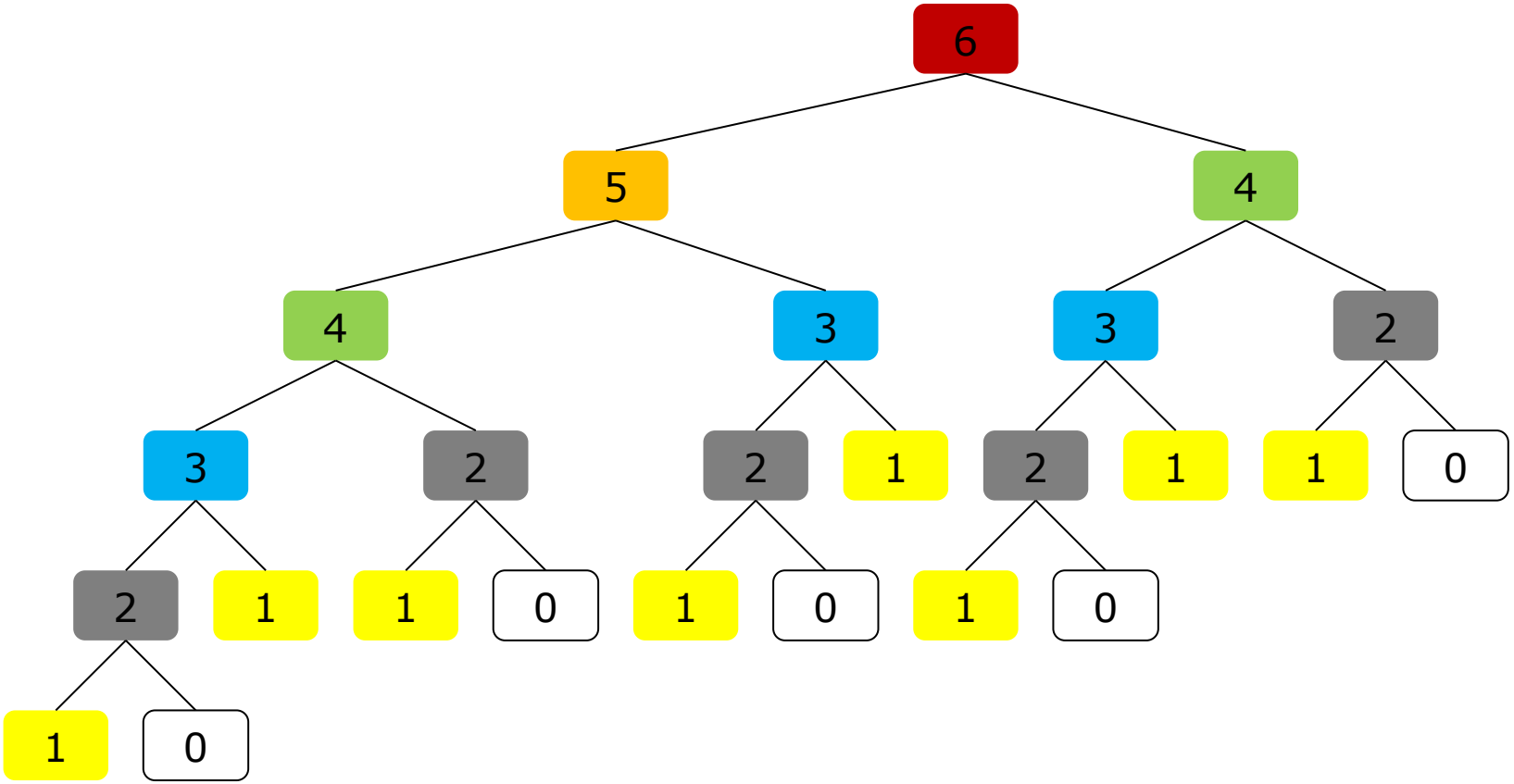
Parallel Version of Fibonacci


```
int fibonacci(int n){
  if(n<2)
    return n;
  int a, b;
  #pragma omp parallel sections
  {
    #pragma omp section
    {a=fib(n-1);}
    #pragma omp section
    {b=fib(n-2);}
  }
  return a+b;
}
```

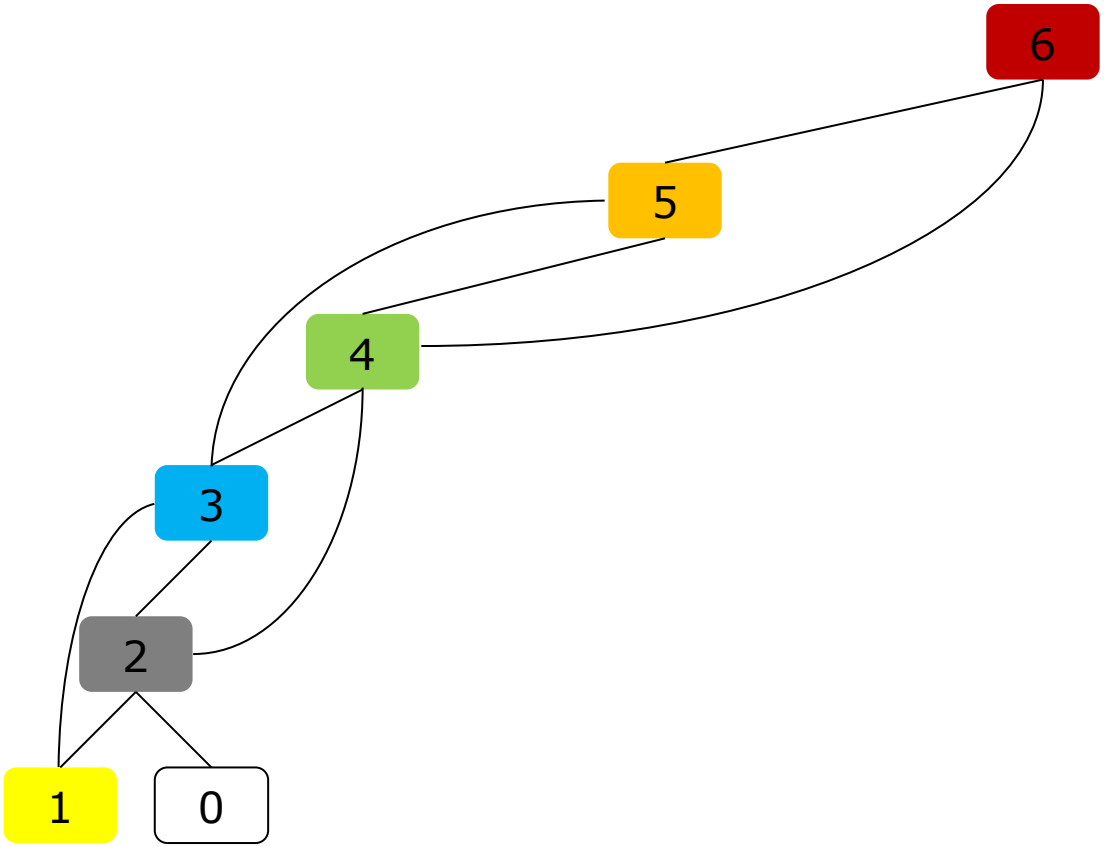
Parallel Version of Fibonacci



Parallel Version of Fibonacci



Linear Sequential Version of Fibonacci



Linear Sequential Version of Fibonacci


```
int fib (int n){
  int prev1=0, prev2=1;
  int i=0;
  for(i=0;i<n;i++){
    int savePrev1 = prev1;
    prev1 = prev2;
    prev2 = savePrev1+prev2;
  }
  return prev1;
}
```

Better than parallel:

- No repeated operations
- No overhead

Understand the Problem

1. Analyze the data dependences of the problem to identify if it can be parallelized
   - Example: chain of dependences in Fibonacci number computation prevents from efficiently parallelize it
2. Evaluate different sequential algorithms
   - Most of parallel algorithms are formulated starting from sequential versions
   - Design a parallel algorithm completely from scratch can be a very hard task

Understand the Algorithm

- Identify hotspots
  - Know where most of the real work is being done
  - Focus on parallelizing these parts
- A hotspot can be a bottleneck if it can not be parallelized
  - Identify if this part actually can not be parallelized (e.g., reading a sequence of input)
  - Analyze if this part can be restructured in order to be parallelized

Automatic tool support

- Design of parallel algorithms and developing of parallel programs is a mainly manual activity
- Automatic tools can however help the designer in:
  - identifying hotspots
  - analyzing sequential implementations to provide hints on the parallelization
  - verifying correctness of produced solutions
    - E.g.: verifying that parallel tasks actually do not write the same data
  - estimating the performance of produced solutions

References

- Material prepared by Serena Curzel and Marco Lattuada for previous editions of this course
- I.Foster, “Designing and Building Parallel Programs”, Addison-Wesley, 1995 (available online)